

A G U I D E D I N T R O D U C T I O N

ALBEDO

Version Beta

*Everything the system is, how each part of it works, and why any
of it was built this way — written for a reader who has never
opened a terminal.*

A L K M Y

2 September 2026 · build 0.1.0-alpha.1 · commit 668f33e

How to read this

You will not be asked to write a line of code. Nothing here needs it. What follows is a walk through a machine, starting from the pavement outside and ending in the room where the decisions get made, and the only thing it asks of you is that you read the parts in order — because each one is built out of the one before it.

The first part contains no Albedo at all. It explains what a website is, what a browser does, and what the last fifteen years of web development actually cost, in the plainest terms I can manage. If you already know some of it, skim. But the second part will not land without it, because Albedo is an answer, and an answer is meaningless until you have felt the question.

The middle parts take the system apart. Every subsystem has a name — FORGE, APERTURE, SANDGATE, PRISM, PHOSPHOR, SHUTTER — and each name covers one honest idea that you can hold in your head. I will give you the idea before the machinery.

The fourth part is about speed, and it is long on purpose, because speed is the thing people will ask you about and it is the thing most easily faked. Every number in it comes from a measurement someone ran on a specific machine on a specific day, and where a number was later shown to be wrong, that is written down too.

The last part is for you rather than for the system: a glossary, a map of the repository, and the handful of questions you will be asked in a room, with what to say.

ONE PROMISE ABOUT THE NUMBERS

Every figure in this document was taken from the source code, from a test that runs, or from a measurement file in the repository — and each one is stated with the thing it was measured on. Where a claim has been tested and *failed*, that failure is recorded here too. There is one number in the product that is fabricated for demonstration purposes, the output of the `albedo demo` command, and it appears nowhere in these pages.

The version this describes is `0.1.0-alpha.1`, at commit `668f33e`, on 1 September 2026. Roughly 132,600 lines of Rust across 215 files, with 1,950 tests passing.

· · ·

Contents

Part	Chapters
One	The ground. Two computers and a conversation · the page that arrives finished · the program that arrives instead · what a framework is · who pays the bill
Two	The idea. One sentence · what a compiler is · the three tiers · who owns the state · the two impossibilities

Part	Chapters
Three	The machine. The pipeline · the parser and the graph · the manifest · two renderers · the wire · PHOSPHOR · PRISM · FORGE · APERTURE · AUTH · SANDGATE · SHUTTER · the three refusals · npm · the commands
Four	Speed. Four clocks · the language choice · the request, itemised · not shipping the work · measuring the shape · the refutation we published · the data structures · O of the change · what is still slow
Five	What it means. The honest ledger · where it competes · glossary · the map of the repository · five questions and what to say

The Ground

*No Albedo in this part. Just the world it was built into, and the bill
that world has been quietly running up.*

CHAPTER 1

Two computers and a conversation

A website is a conversation between two computers that have never met and will forget each other immediately.

One of them is in front of you. Your laptop, your phone. Running a program called a *browser* — Chrome, Safari, Firefox. The other one is somewhere else entirely: a rented machine in a building in Virginia or Mumbai, switched on all the time, waiting. That one is called the *server*, and the only thing it does all day is wait to be asked something.

You type an address. The browser sends a short message to the server that means, roughly, *give me the thing at this address*. That message is a **request**. The server thinks for a moment and sends something back. That is a **response**. Then both sides forget the whole exchange happened.

That forgetting is important, and we will come back to it more than once.

The forgetting is why a website has to be told who you are on every single request, why staying logged in is a genuine engineering problem rather than an obvious one, and why a page that needs to update itself when someone else does something has to keep a line open — because otherwise nobody is listening.

Everything else in this document is detail hung on that one exchange. Something is asked for. Something comes back. What comes back, and how much work the receiving end has to do to make sense of it, is the entire subject.

CHAPTER 2

The page that arrives finished

What comes back from the server, in the oldest and simplest version of the web, is a document. It is written in a notation called **HTML**, and HTML is just text with labels on it.

```
<h1>Guestbook</h1>
<ul>
  <li>ada – first light</li>
  <li>alan – the machine stirs</li>
</ul>
```

A browser reading that knows `h1` means a heading, `ul` means a list, `li` means one item in it. It draws them. Nothing has to be computed, decided, or assembled. The document arrived already being the thing it describes.

A page like that is fast in a way that is hard to improve on, because there is no work left to do. The bytes arrive, the browser paints, and it is finished. On a slow phone on a bad connection in a train tunnel, it still works, because the only requirement is that the text arrives.

It has one obvious limitation. Nothing on it moves. Press a button and nothing happens, because there is no button — there is a rectangle that a browser drew, with no instructions attached about what to do when it is pressed.

So the industry added instructions. That is where the bill starts.

CHAPTER 3

The program that arrives instead

The instructions are written in a language called **JavaScript**, and a browser can run them. That capability is the reason the modern web exists: a page can now respond, change, animate, fetch more data without reloading, and behave like an application rather than a poster.

The way that capability got used, though, took a particular turn around 2013 and never turned back.

Rather than sending a finished page with a few instructions attached, the industry moved to sending an *empty* page plus a program that would build the page once it arrived. The server stopped answering “here is your guestbook” and started answering “here is a program that, when you run it, will ask me for the guestbook and then draw it”.

You can watch this happen. A page loads, is blank or shows a spinner for a beat, then fills in. That beat is the browser downloading the program, parsing it, running it, discovering what it needs, asking for it, waiting, and finally drawing.

Then it got a name: hydration

Eventually people noticed the blank beat was bad, and invented a compromise. The server would render the finished HTML *and* send the program. The HTML shows up immediately, so the page looks ready. Then the program downloads, runs, rebuilds the same page invisibly in its own memory, compares it to what is already on screen, and takes ownership of it so that the buttons start working.

That second step is called **hydration**, and it is the central cost of the modern web. The page is drawn twice — once by the server in HTML, once by the browser in memory — and the second drawing exists only to make the first one clickable.

THE PART WORTH HOLDING ON TO

Hydration means the visitor downloads a program that reconstructs a page they can already see. The work is real, it happens on their device, on their battery, and it happens on **every component of the page** — including the ones that will never move, never be clicked, and never change for as long as the page is open.

A page of a hundred components where three of them are interactive still hydrates all hundred.

CHAPTER 4

What a framework is, and why there are so many tools

Writing that program by hand is miserable, so libraries appeared to make it manageable. The dominant one is **React**, made at Facebook. React lets you describe a piece of the page as a *component* — a self-contained thing with a name, like **Counter** or **Guestbook** — and it works out how to keep the screen in step with the data.

React by itself does not know about servers, addresses, or files. So a second layer appeared on top of it: **Next.js**, made at a company called Vercel, which adds routing, server rendering, and a build process. Next.js is the default choice for most new web applications today, and it is the thing Albedo is measured against in every conversation you will have.

Underneath both sits **Node.js**, which is a way of running JavaScript on a server instead of in a browser, and beside them sit the *bundlers* — tools called Webpack, Vite, esbuild, Turbopack, whose job is to take the several thousand files a modern project consists of and squash them into a handful the browser can actually download.

And then there is **npm**, the shared warehouse. Roughly three million published packages of other people’s code, which any project can pull in by name. Almost nothing is written from scratch any more; a normal application is a thin layer of your own code sitting on a few hundred imported ones.

Add it up and a typical modern web project is a stack of independent tools that each have to be installed, configured, kept in version, and understood: a language, a runtime, a library, a framework, a bundler, a package manager, a database, an ORM, a migration tool, an authentication library, a hosting adapter.

Each of those was a reasonable idea. Together they are a career.

CHAPTER 5

Who pays the bill

There are two bills, and they land on two different people.

The **developer** pays in configuration. Learning eleven tools, keeping them agreeing with each other, and diagnosing failures that live in the gaps between them. This is the bill everyone complains about, and it is the one the industry has spent a decade trying to reduce.

The **visitor** pays in bytes and milliseconds, and almost nobody sees this bill because the people building the site are testing it on a fast laptop on office wifi. Someone opening the same page on a four-year-old Android phone on a patchy connection is downloading several hundred kilobytes of program, parsing it, and running it, so that a page they could already see becomes clickable.

That second bill is the one Albedo is aimed at. Not because the first one does not matter — Albedo reduces it too, and substantially — but because the second one is the one that is structurally hard to fix, and the one that no amount of better tooling has managed to touch.

*The whole question is: how much of this page did we send a
program for, and how much of it did we simply send?*

THE QUESTION ALBEDO EXISTS TO ANSWER

The Idea

One sentence, and then the four consequences that fall out of it.

CHAPTER 6

The sentence

Albedo reads your entire application before it ever runs, decides for each piece of the page where that piece should live, and then emits exactly what that decision requires — and nothing else.

That is the whole product. Everything in Part Three is machinery for making that sentence true and keeping it true under pressure.

Compare it to the alternative. In a conventional setup, the developer decides. Next.js has a marker, "use client", that a developer writes at the top of a file to say *this one needs to run in the browser*. It is a human judgement, made file by file, usually under time pressure, and it is wrong all the time — in both directions. Mark too much and you ship a program nobody needed. Mark too little and the page is broken.

Albedo removes the marker. The decision is derived from what the code demonstrably does, by a program that has read all of it, is not tired, and does not have a deadline.

CHAPTER 7

What a compiler actually is

A compiler is a translator that reads a whole text before it writes anything. That second half is what makes it interesting.

A phrasebook translator works word by word and can only do what the words in front of it permit. A translator who has read the entire novel first can make decisions no word-by-word process can: that a character's name should be rendered differently in chapter nine because of something in chapter two, that a joke on page forty is set up on page eleven and both must move together.

The Albedo compiler reads your whole application: every component, every import, every use of state, every handler, every reference to your database and to the outside world. It builds a picture of how the parts relate. And only then does it decide what to emit.

It can therefore know things a developer typing in a single file cannot easily know — that this button’s click handler eventually reaches the database, that this piece of state is also read by another visitor in another country, that this component names the logged-in user and therefore cannot be baked ahead of time.

The compiler lives in `src/`, is written in a language called Rust, and its entry point is a type called `RenderCompiler`. It has one job title and a lot of departments.

CHAPTER 8

The three tiers

Every component in an Albedo application is sorted into one of three tiers. The sorting happens at build time, once, in a single function — the tier decision — which lives in one file, `src/effects.rs`, and is about two hundred lines long.

Tier	Who owns the state	What is sent to the browser
A	Nobody. There is no state.	Markup. No program of any kind.
B	The server. The state provably leaves this visitor — it is shared, stored, or computed on the server.	Markup. Still no component program. Updates arrive later as small instructions on an open line.
C	The browser. The state stays with this one visitor.	A compiled <i>island</i> — this component’s code alone, not the page around it.

A **Tier A** component is something like a page heading, a footer, a paragraph of marketing copy. It has no memory and nothing to react to. Albedo renders it once, at build time, and stamps the result into the artifact. It costs the visitor the bytes of its own text and nothing more.

A **Tier B** component is the interesting one, and it is the tier that does not exist in any competing system. Think of a chat room, a live list, a guestbook. It changes — but the thing driving the change is on the server, because other people are causing it. So the server renders the markup, holds the state, and when the state moves, sends the browser a short instruction saying which piece of the page to change and what to change it to. The component’s own code never crosses the wire.

A **Tier C** component is a counter, a dropdown, a form with local validation. Its state belongs to the person looking at it and to nobody else. There is no honest way to run that on a server, so Albedo compiles that component — that one, on its own — into a small program and sends it. This is called an *island*: a piece of live code surrounded by static markup.

The number, and the caveat that comes with it

On 5 August 2026 this classifier was run against 1,398 real components taken from five real open-source projects that had never heard of Albedo — `bulletproof-react`, `shadcn-taxonomy`, `vercel-commerce`, `excalidraw`, and Cal.com’s web application. Zero parse failures.

Tier	Count	Share	Meaning
A	429	30.7%	markup only
B	427	30.5%	markup only; updates ride the wire
C	542	38.8%	becomes an island

Measured 2026-08-05 by the tiering corpus probe in `tests/`, running the production classifier on the production inputs. Recorded in the tier distribution file under `development-plan/`.

61.2% of real components ship no component JavaScript at all. That is the headline, and it is the number to quote, because it was measured on code written by strangers.

THE CAVEAT, WHICH YOU SHOULD VOLUNTEER BEFORE ANYONE ASKS

The same harness was run against Albedo’s own sample code — the starter project, the demo app, the test fixtures. It came out at **55.8% Tier A**, against 30.7% on real code.

Twenty-five points of flattery. Our own code is written by people who know what the classifier likes, so of course it classifies well. The measurement harness deliberately refuses to print a combined figure of the two, so that nobody can accidentally quote the good one.

If you ever hear an Albedo “zero JavaScript” claim rehearsed against our own starter project, it is twenty-five points optimistic. Say so.

Which rule actually decides

The same measurement recorded *why* each component landed where it did. One rule dominates.

Reason	n	Share	Note
HookDrivenHydration	555	39.7%	the component holds memory
PureStaticEligible	429	30.7%	exactly the Tier A set
SideEffectBoundary	185	13.2%	forced to Tier C, no escape
AsyncBoundary	155	11.1%	waits on something
WeightBasedPromotion	57	4.1%	simply too large
IoBoundary	17	1.2%	touches the outside world

CHAPTER 9

Who owns the state — the question that turned out to be the right one

This chapter is a story about being wrong, and it is the single best illustration of how Albedo is built, so it is worth your time.

The original rule for separating Tier B from Tier C was *is this component interactive in a way the browser can satisfy on its own?* If yes, ship an island. It sounds correct. It was the design for months.

Then somebody measured it. Across the 1,398-component corpus there were 536 interactive components. The rule answered *yes* for **534 of them** — 99.6%.

A test that gives the same answer 99.6% of the time is not a test. It is a constant wearing a test's clothes.

Worse, the follow-up investigation on 7 August found the reason, and the reason was structural. The analysis that computes that bit runs one file at a time, during parsing, before the compiler has resolved what the file's imports point at. So a click handler that reaches the server *through an import* is invisible to it. The bit almost never goes false because the evidence it needs is in a file it is not looking at.

The replacement rule asks something different: **does this state provably leave this visitor?** Is it written into a shared channel, stored in the database, sent to a server action, or pushed across a network boundary? If it leaves, the server owns it, the update has to travel to everybody anyway, and no island is needed. Tier B.

And that rule is consulted *first*, before the old one — deliberately, with a comment in the source explaining why. A shared counter with a click handler satisfies both rules. The old ordering saw the first fact only, and shipped a program for state the wire was already carrying.

THE ASYMMETRY THAT GOVERNS THE WHOLE CLASSIFIER

When the compiler cannot prove which way a component goes, it rounds toward **Tier C**.

A component wrongly placed at Tier C still works — it ships a program that was not strictly necessary, which is waste. A component wrongly placed at Tier B ships a binding for state the wire cannot actually drive, and the page is silently broken.

Waste is recoverable. Silence is not. Every ambiguous case in the tier decision resolves toward the recoverable failure, and the source says so at each one.

CHAPTER 10

The two impossibilities

Most of the tier rules are preferences. Two of them are not — they are statements that a tier is physically unavailable, and both were discovered by shipping the bug first.

A component that names the logged-in user cannot be Tier A

Tier A markup is baked at build time, on a build machine, weeks before anybody visits. There is no visitor at that moment. So a component that says *if there is a user, greet them, otherwise show a sign-in link* gets baked with no user — and renders the signed-out branch to everyone, forever. The signed-in branch is not merely unreachable; it is absent from the artifact.

The failure is silent, which is what makes it dangerous. Nothing errors. The page returns a perfectly healthy response with the wrong content in it. The compiler now reads the component's source looking for the word `user`, and a component that names it is sent to Tier B — the only tier that has a live request to read the answer from. The reason is recorded as `RequestScoped`.

A component that imports an npm package cannot be Tier A either

This one fails loudly rather than silently, and it fails worse.

Tier A markup is produced by a small pure-Rust evaluator that walks the code directly. That evaluator can resolve a reference to a file next door; it cannot resolve a reference to a package from the npm warehouse. So the component raises an error.

And here is the part that took a real incident to learn: a Tier A render is **one call over the whole subtree**. When a leaf raises, the error travels all the way up, and every ancestor's markup is discarded along with it. One unresolvable import in a small component at the bottom of the page deletes the page.

So the npm-import fact is checked, and the component is sent to Tier B — where a full JavaScript engine is available on the server and renders it correctly. Notably it is *not* sent to Tier C, even though Tier C is where npm demonstrably works. Tier C is the tier that ships code to the browser, and this component asked for none of that. It has no memory, no effects, no handlers. Promoting it would buy a correct render by shipping a program for a page that needs zero — and “how much did we not ship” is the entire product. The comment in `src/effects.rs` that argues this out runs to fourteen lines.

*A wrong Tier C is expensive. A wrong Tier B is a lie. A wrong
Tier A can delete the page.*

THE THREE FAILURE MODES, RANKED

The Machine

Sixteen chapters, one subsystem each. Each has a name, one idea, and a reason it was built the way it was rather than the obvious way.

CHAPTER 11

The pipeline, end to end

When someone types `albedo build`, nine things happen in order. Every subsystem in the rest of this part attaches to one of these nine steps, so it is worth having the shape in your head first.

1. **Scan.** Walk the project folder and find every source file. This is `src/scanner.rs`.
2. **Parse.** Turn each file's text into a structure the compiler can reason about.
`src/parser.rs`, built on a parser called SWC.
3. **Build the graph.** Work out which component uses which. `src/graph.rs`.
4. **Analyse.** Give every component an *effect profile* — does it hold memory, wait on things, touch the outside world, cause side effects — and an estimated weight.
`src/analysis/`.
5. **Decide.** Run the tier decision on each component. This is the moment the product happens.
6. **Schedule.** Sort the graph so independent work can proceed in parallel, and find the longest chain of dependent work — the *critical path*. `src/analysis/`.
7. **Emit the manifest.** Write down every decision in a single file, `RenderManifestV2`.
`src/manifest/`.
8. **Bundle.** Classify → plan → rewrite → emit. Produce the island programs, the npm chunks, the static markup. `src/bundler/`.
9. **Check.** Run the preflight rules, the tier budget, and refuse to finish if anything is wrong. `src/preflight.rs`, `src/budget/`.

Then `albedo serve` picks up the artifact and starts answering requests. The server is a separate crate, `crates/albedo-server`, about 5,800 lines in its main file, built on a Rust web framework called Axum.

There is a fast path for development. `albedo dev` runs the same pipeline but keeps an **incremental cache** — `src/incremental.rs` — which stores a hash of every file and the analysis it produced. On a save it re-analyses the changed file and everything downstream of it, and reuses the rest.

CHAPTER 12

Reading the code: the parser and the tree

A compiler cannot reason about text. It has to convert the text into a structure first, and that structure is called an **abstract syntax tree**.

It is the same idea as a sentence diagram from a grammar lesson. *The cat sat on the mat* is a string of letters until you draw it as a subject, a verb, and a prepositional phrase — at which point you can ask questions of it, like “what is the subject”, that the string of letters could not answer.

Albedo uses a parser called **SWC** for this. SWC is written in Rust and is the same parser Next.js uses internally, which matters more than it sounds: it means Albedo is reading the code with exactly the tool the ecosystem’s dominant framework reads it with, so an obscure syntax that works there works here.

Having the tree, the compiler runs a series of small passes over it. These live in `src/transforms/` and each one has one job:

Pass	What it surfaces or rewrites
hooks	every <code>useState</code> — a piece of component memory — in source order, because the order is the contract
events	every <code>onClick</code> and its ilk, keeping the handler’s body intact so the server can re-run it
forms	every <code><form action="action:name"></code> , and the field names inside it
actions	every server action the author declared with <code>action(...)</code>
links	every <code><Link href></code> , so navigation can be intercepted rather than reloading the page
shared slots	every <code>useSharedSlot(...)</code> — the hook that reads live server-owned data
shared slot lists	the list-shaped case of the above, which needs different machinery
CSS modules	scopes a stylesheet’s class names so two files cannot collide

Pass	What it surfaces or rewrites
stable ids	stamps an addressable identifier onto every element in the markup
deferred children	defers a component's children so the parent runs first
workflow	rewrites <code>await</code> inside a handler so it can run on an engine with no waiting

Two of those passes deserve their own paragraphs

The deferred-children pass fixed a bug that had been in the system from the beginning and was invisible until Radix — a widely used component library — was tried against it. JSX, the notation React components are written in, turns `<Wrapper><Inner /></Wrapper>` into a function call with `Inner` as an argument. JavaScript evaluates arguments before the call. So the server ran `Inner` first and `Wrapper` second, which is the exact reverse of the order React uses.

That single ordering fact caused two separate failures. Element identifiers came out transposed between the server and the browser, so the browser silently rewrote every one the server had baked in. And a component that provides context to its children could not possibly do so, because the children were already finished HTML by the time the provider ran. Real Radix did not degrade gracefully; it threw an error, the render failed, and the component was simply absent from the page.

The fix wraps each child in a deferred call, so it is not evaluated until the point where it is actually placed — which is after the parent's body has run. Radix went from 3 of 12 components rendering to a working set of **six gated packages**, rendered on the server with no browser JavaScript at all.

A PHRASING NOTE YOU WILL NEED

A commit message in the repository says “20/20”. That number came from an ungated sweep and is not what the test suite pins. The defensible claim is the six gated Radix packages. Use that one.

The stable-ids pass closed a different hole. Two different renderers produce markup in this system, and one of them was stamping addressable identifiers onto elements while the other stamped nothing. The instruction stream then referred to element identifiers that existed nowhere in the document, the browser could not find them, and the whole batch of instructions was abandoned. No Tier B click handler had ever bound.

It survived undetected because the only two Tier B components anybody had written were forms, and a form binds by attribute rather than by identifier. The one component shape that would have exposed it was the shape nobody had happened to write.

CHAPTER 13

The graph, the critical path, and the four lanes

Components use other components. The page uses the header, which uses the logo. Written down, that is a graph — dots joined by arrows.

Albedo builds that graph and then sorts it, so that work with no dependency on other work can be done at the same time. Then it finds the **critical path**: the longest chain of things that must happen one after another, because that chain sets the floor on how fast the whole build can possibly be.

The scheduler then assigns components to **four lanes**. Four is a fixed constant, and the internal data layout is physically aligned to it — there is a compile-time assertion in `src/runtime/highway.rs` that fails the build if the two lane counts ever drift apart. Lanes let rendering fan out across processor cores, and let updates be routed without contention.

The critical-path walk is also the subject of the best optimisation story in the repository, which is told properly in Part Four.

CHAPTER 14

The manifest — the build's written record

Everything the compiler decided is written into one artifact called `RenderManifestV2`. Every component, its tier, its hydration mode, its weight, its data dependencies, its route, its slot identifiers. The server reads this file and nothing else about your source code.

That separation is deliberate and load-bearing: the server has no compiler in it. It cannot re-decide anything at runtime, which means a running production server cannot drift from what the build proved.

The manifest also carries the route table. Albedo uses **file-based routing** (`src/routing/`), which means the folder structure is the address structure — a file at `src/routes/guestbook.tsx` serves the address `/guestbook`, and a file at `src/routes/room/[id].tsx` serves `/room/lobby`, `/room/quiet`, and anything else in that position. The square brackets mark a *parameter*.

A STUB THAT LIED FOR MONTHS

The manifest carries a field listing the server actions a route is allowed to invoke. It is one of three doors on the route gate.

It was emitted as an empty list. In every manifest the compiler had ever produced. The gate had three doors and enforced one, and nothing in the test suite noticed, because every test constructed its own manifest by hand rather than asking the compiler for one.

The rule that came out of it is now written down: **at least one test per gate must start from what the compiler actually produces.**

CHAPTER 15

Two renderers, and the discipline of agreement

Turning a component into markup happens in two different ways in Albedo, for two different reasons, and keeping them in agreement is a permanent engineering obligation.

The first is a **pure Rust evaluator** — `src/runtime/eval/` — which walks the syntax tree directly and produces HTML. It is fast, it has no engine underneath it, and it is what bakes Tier A markup at build time. It cannot resolve npm packages, which is exactly why npm forces a component out of Tier A.

The second is **QuickJS**, a small complete JavaScript engine written in C by Fabrice Bellard, embedded into the Rust binary through a wrapper called `rquickjs`. This is a real engine — it can run arbitrary JavaScript, including three million npm packages' worth of it. The QuickJS module in `src/runtime/` is the largest single file in the project at 6,633 lines. It serves every Tier B render.

And there is a third renderer that is not in Rust at all: `assets/albedo-client.js`, the browser-side runtime that owns Tier C islands once they arrive.

Three implementations of the same semantics is a standing invitation to divergence, so the project keeps a **conformance harness** (`src/conformance/`, and a conformance suite in `tests/`) whose job is to run the same input through each and compare the output byte for byte.

A WARNING ABOUT THAT HARNESS

One of its reports says `declined: 0`, which reads like “nothing failed”. It is a fact about the corpus rather than about the evaluator — every fixture in the tree was written against what the system could already do, so of course it passes.

This exact trap has now caught the project twice: once here, once with the tier classifier’s 25-point flattery. It is now a rule of the house that a control corpus cannot support a claim about real-world code.

CHAPTER 16

The wire — what travels when something changes

A Tier B component ships no program, so when its data changes the browser has to be told what to do in a language it already understands. Albedo’s answer is a small instruction set — **opcodes** — defined in `src/ir/opcode.rs`.

The instructions are exactly what you would guess: `Create` an element, `SetText`, `SetAttr`, `Append`, `Remove`, `BindEvent`, `BindSlot`, `Placeholder`, `Patch`, plus a set for reconciling lists — `SlotSet`, `SlotDelta`, `SlotInsert`, `ReconcileList`. About twenty in total.

They are encoded in a compact binary format called **bincode**, not in text, and they refer to elements, tags and attribute names by number rather than by name. The number-to-name table is called an **intern table**, and it is sent once and then patched, so a page that mentions the tag `div` two thousand times sends the word once.

The internal representation the compiler holds these in is *columnar* — `src/ir/columns.rs`, `IrColumns`. Instead of a list of records each holding several fields, it keeps one array per field. This is a well-known layout for making processors fast, and it is discussed in Part Four.

The measurement that changed the wire

Until 23 July 2026, a change to a list shipped two things: the authoritative new value of the whole list, and then the small instruction describing what actually changed. Belt and braces.

Somebody measured it on a guestbook with 2,002 rows. Appending one row sent **126,023 bytes** of full-list snapshot beside a **179-byte** delta.

Seven hundred and four times the necessary payload. On every write. To every connected subscriber. Base64-encoded again by each connection on the way out.

The rule now is a single sentence: the snapshot ships only when nothing else on the frame carries the change. A list instruction carries it, so the snapshot is dropped; a plain value has no list instruction, so the snapshot is the frame and the wire is byte-identical to what it was before. The stored value on the server is untouched either way, so a page reload, a new subscriber, or a resync still gets the whole truth.

CHAPTER 17

PHOSPHOR — one connection per browser, not per tab

For those instructions to arrive, a line has to stay open between the browser and the server. The first design gave every tab its own line.

Browsers cap how many simultaneous connections they will open to one address — around six. A live line consumes up to two of them. So three tabs of the same application exhausted the browser's own pool, and the next request the page tried to make simply queued.

It was measured. A plain page request sat waiting for **27,361 milliseconds** — twenty-seven seconds — and completed the instant another tab was closed.

PHOSPHOR restructures it. One browser profile holds **one** physical connection, called the *trunk*. A tab holds zero. What a tab holds instead is a *circuit*: a route-scoped, server-authorised, reference-counted subscription that rides the shared trunk. Connections become proportional to the number of browsers rather than the number of tabs, and tabs are free.

Three properties are enforced there, and each closes a specific failure:

- **The server decides which channels a tab is subscribed to, never the client.** A tab asks to subscribe to a route; the server resolves that route through the same router and manifest the render used. All of it goes through one choke point, a type called `RouteAuthority`.
- **The initial state always precedes live updates.** Guaranteed by construction rather than by care: a circuit's buffer can fill from the moment it registers, but nothing enters the trunk until the seed has been pushed, and the trunk is first-in-first-out.
- **A lane that cannot keep up dies loudly.** If the trunk backs up, the whole lane is killed and the client reconnects with a resync. Dropping one circuit quietly would leave some routes stale on a live connection — a page that looks current and is not, which is the failure this whole area exists to prevent.

CHAPTER 18

PRISM — why you cannot spell your own channel name

Live data flows on named channels called **topics**. `guestbook` is a topic. So is the chat room `messages:room=lobby`.

An author writes `useSharedSlot(messages.where({ room: params.id }))` and never types a channel name. That `.where(...)` is not a function that runs; it is a marker the compiler reads. The channel identity is *minted* by the compiler from the collection and the key, in one function — the topic minter.

The reason authors are not allowed to spell it is not tidiness. A hand-written channel name admits two logically distinct partitions colliding on one channel and cross-delivering each other's rows — one tenant seeing another tenant's data, and it would look completely normal in a browser. Minting the name makes that collision *unexpressible* rather than merely forbidden.

The same rule solves access control for free, which is the elegant part.

NAMING THE CHANNEL IS THE ACCESS CONTROL

A session belonging to user `7f3a` can name the topic `todos:owner=7f3a`. A session belonging to anyone else cannot construct that name, because the name is minted from the request's own identity by the server.

So there is no authorization rule written anywhere in the application for per-user data. There is nothing to forget to write, and nothing to get wrong.

The rule that carries this: **a signed-out visitor resolves an identity-keyed channel to nothing**. Not to an empty key, not to an `"anonymous"` placeholder, not to the collection's unpartitioned value. A shared fallback would be one namespace holding every anonymous visitor's rows, and it would look entirely normal.

Three separate parts of the system must derive the same channel identity — the render, the subscription, and the write. Each deriving it independently is individually plausible and the composite failure is *the row appeared for nobody*. So the resolution rule lives in exactly one module, `src/runtime/topics.rs`, and is reimplemented nowhere.

One more rule: a channel that cannot be resolved — a malformed key, a parameter the route never matched — produces **no channel**, rather than a failed render. A strange identifier in a URL must not take the page down. The route degrades to a static page with an empty slot, which is recoverable by fixing the address.

FORGE — the backend that is a declaration

The engine already decides where your interface runs. FORGE teaches it to decide where your data lives, so the backend stops being a system you integrate and becomes an artifact the compiler emits.

In a normal project, adding a guestbook means: designing a table, writing a migration, installing an ORM, writing a query, writing an API endpoint, writing a client that calls the endpoint, and writing something to keep other people's browsers in step.

In Albedo it means adding this to the configuration file:

```
forge: {  
  guestbook: {  
    fields: { author: "text", message: "text" },  
    seed: [  
      { author: "ada", message: "first light" },  
      { author: "alan", message: "the machine stirs" },  
    ],  
  },  
},
```

From that, Albedo emits the table, the query that materialises it, the seed rows, and a live channel that every connected browser can subscribe to. A field is `text`, `int`, `real`, `bool` or `timestamp`, with a trailing `?` for optional. An `id` column is implicit on every collection and is what live reconciliation keys on.

The storage underneath is **libSQL**, a fork of SQLite. But the whole storage layer sits behind one interface, `DataSubstrate` (`src/forge/substrate.rs`), so nothing above it knows what it is talking to — the in-memory test double, the libSQL implementation, and a future Postgres or edge key-value store all satisfy the same seam.

Why every read goes through a declared collection

There is deliberately no way to hand a raw query to a component. That sounds like a limitation and it is the thing that makes everything else possible: because every read reaches a caller through a *declared collection*, every read is a topic; because it is a topic, it can be partitioned; because it can be partitioned, it can be keyed by identity. There was once a variant in the code that allowed a raw query, and it was removed, with the reasoning kept next to the gap it left.

Schema drift, and the refusal

Editing a collection that already holds rows is where data gets destroyed in most systems. FORGE's boot gate, in `src/forge/drift.rs`, reconciles the declaration against the actual database, and it does exactly two things.

It adds new *optional* columns, and says so. Everything else — a drop, a rename, a type change, a new required field — refuses to start and names the field, because there is no value it could invent for the rows that were written before the field existed. A refusal never touches your data.

Reservations, or how not to sell the same seat twice

`src/forge/reserve.rs` is the contention primitive: atomic claiming of a bounded resource — tickets, stock, seats, quotas. Supply never goes negative, nothing is ever oversold, and a request that is retried never claims twice. This is the class of problem that is easy to get 95% right and catastrophic to get 99% right, and it is 903 lines of code with its own concurrency test file.

The write path, and the shape of a change

A write goes: mutate, commit, re-materialise, fan out. The fan-out happens strictly *after* the transaction commits — broadcasting from inside would let subscribers observe a state that a failed commit then erases, a value that never existed.

What is fanned out is a **z-set delta**: a signed multiset of the records that differ. **+1** inserts a row, **-1** retracts one, and an edit is both under the same key. Appending one row to a two-thousand-row list costs the size of the row.

THE RULE THAT KEEPS DELTAS HONEST

Every uncertainty in the delta path resolves to **no delta**, never to a guessed one. Rows with no usable key, duplicate keys, a previous value that is not a list, a row the renderer declines — each yields nothing, and the system falls back to shipping the whole snapshot.

That fallback is slower and always right. A *wrong* delta is silent and permanent: the browser applies it, diverges from the server, and shows a page that no further interaction will ever correct.

The asymmetry is total, so the rule is total.

One structural detail worth knowing: the thing that turns a record into rendered markup is handed in from the render side as a parameter. FORGE never learns what an `<li>` is. And the renderer is asked to render the *whole* view rather than one row — because a row template is not a pure function of its record. It may close over the row's index, read the list's length,

or depend on what the surrounding component computed. Rendering one record in isolation produces markup that *almost* matches a reload, and “almost” here means a page that silently disagrees with the server.

CHAPTER 20

APERTURE — the outside world as a tier

A remote resource is a channel whose derivation is an HTTP request. A remote write is a journal entry whose position is its idempotency key.

Applications talk to other people’s services — Stripe for payments, a weather API, an internal service. APERTURE is the subsystem that lets Albedo do that without giving up any of the guarantees the rest of the system makes.

The read side is the easy half. A declared `source` is a channel like any other; a component subscribes to it with the same hook it uses for the database; there is a shared response cache with an eviction budget, request coalescing so a hundred simultaneous requests for the same thing become one, and conditional revalidation so an unchanged response costs almost nothing.

The write side, and the double charge

The hard half is what happens when your server crashes in the middle of charging somebody’s card.

Every non-deterministic act a handler performs is recorded in an append-only log called the **journal**. Its design decision, described in the source as the single load-bearing one, is that it is a log with a stable step index rather than a lookup table — because *the position of a step in the log is the idempotency key of the request it records*. A retry lands on the same position and therefore presents the same key, and the upstream service recognises it as the same request.

The obvious way to make that log durable is to write down what happened after the call returns. That misses the only window it exists for.

The write has to bracket the call, not follow it.

So: record the step as `Unknown`, on disk. Issue the request. Record the outcome. A crash inside the bracket leaves an `Unknown` row on disk — and that is a row, not a gap. The replay finds it, keeps the step index, and reissues under the same derived key. The retry is a retry rather than a second charge.

The identity of a step is derived from the action, the principal, and the intent, rather than from a fresh random number per dispatch — a random number would make a persisted log unnameable, which was the actual insight the durability work turned on.

This costs **0.504 ms** per outbound call for the bracket, and **0.162 ms** to load a 200-step log, measured on the development machine in release mode against an ephemeral libSQL database. The thing being bracketed is a network round trip — tens to hundreds of milliseconds — so the protection costs a fraction of a percent of what it protects.

AN OPTIMISATION AVAILABLE AND DECLINED

The two halves of the bracket are not equally load-bearing. The opening write must be durable before the call leaves. The closing write need not be — losing it means a completed call is reissued under the same key, which the upstream deduplicates.

So the cost could be halved by making the closing write fire-and-forget. It is declined, on the grounds that it trades a guarantee this server makes for one it would have to *assume* the upstream makes.

Writing ``await`` on an engine that cannot wait

There is a design commitment here worth stating: **there is no Albedo dialect**. An author writes exactly what a vendor's documentation, a Stack Overflow answer, or an assistant hands them.

```
const res = await fetch("https://api.stripe.com/v1/charges", { method: "POST", body
})
const paid = await res.json()
```

That is a requirement rather than a courtesy. A `fetch` that returned a value without `await` would read as uncanny to every engineer who has used the web, and would break every example on the internet.

Two rewrites make it run on an engine with no event loop. The `await` marker is removed at compile time — the engine's `fetch` returns its value directly, because it is answered from the journal, so removing the marker is exact rather than approximate. And every `try/catch` the author wrote gets a guard prepended, because the suspend signal travels as a thrown value and a user's catch block would otherwise swallow it and run on garbage. React's own `Suspense` has this exact bug class.

Egress, and the bypass that was closed by construction

A declared source's address is also its allowlist. Anything the application tries to fetch that was not declared is checked against a set of forbidden address classes — cloud metadata endpoints, link-local addresses, private neighbours.

The naive way to do that check is: resolve the hostname, look at the address, decide, then hand the *hostname* to the HTTP client — which resolves it again. Between those two resolutions the answer can change. A hostile DNS server returns a public address the first time and `169.254.169.254` the second, and walks straight through. This is called DNS rebinding and it is the standard bypass for exactly this control.

So the check is not performed before resolution. **It is resolution.** `ApertureResolver` implements the HTTP client's own resolver interface, which is the only place the client learns an address, and it filters there. There is no second lookup and therefore no window.

An earlier version of this had a real hole: an address typed directly as a number, rather than as a hostname, bypassed the whole thing. That is fixed and pinned by a test.

CHAPTER 21

AUTH — identity as a key

A principal is a compiler-known value, so an authorization policy is never written. It is derived from the read that needs it.

Accounts are declared in the same configuration file as everything else:

```
auth: {  
  session: { ttl: "30d" },  
  providers: { password: {} },  
  login: "/sign-in",  
},
```

That emits four tables — users, accounts, sessions and credentials — mounts the sign-in endpoints, and puts a `user` value on every render, either a principal or nothing. There is no authentication library to install and no users table to design.

Table names beginning with `albedo` are reserved. A collection declared under that prefix is refused at boot, because otherwise an application could replace the session table, or expose every user's row as one live channel.

Two corollaries that are the whole design

Identity is one more key source, beside route parameters. `user.id` mints a channel identity exactly as `params.id` already does. So `todos.where({ owner: user.id })` is per-user live data with no authorization code anywhere in the application — inheriting the guarantee PRISM already gives two chat rooms rather than rebuilding it.

A provider is one more declaration, beside sources. Our password provider and a third party's OAuth provider both produce the same `Principal` type, so application code cannot tell them apart and swapping provider is a configuration edit.

Three decisions in the cryptography, and why each

A session token is 256 bits of cryptographically random output, hashed with SHA-256 before it is stored — so reading the database yields nothing replayable. SHA-256 rather than a slow password hash, on purpose: there is no dictionary to slow down against 256 random bits, and a deliberately slow hash on the every-request lookup path would be a self-inflicted denial of service.

A password is the one input in the system that is low-entropy and guessable, so it is the one that gets the deliberately slow hash: **Argon2id**, OWASP's first recommendation and the only widely available function that resists both graphics-card attacks and side channels. Its version is pinned rather than floated, because a minor update that changes the default cost parameters would silently change every password written from that day on.

The user identifier is minted by Albedo and never adopted from a provider. That is forced rather than chosen: an identifier becomes a channel namespace the moment a component reads `user.id`, and identifiers like `google-oauth2|1234` or an email address fall outside the alphabet a channel key is allowed to use.

What is honestly not finished

Password sign-in works end to end and was proven in a browser. Passkeys, OAuth, delegated identity, and the full reach matrix are designed and not built. And one case is parked with a finding attached, which you should know because it is the most interesting negative result in the auth work.

THE PARKED CASE, STATED PLAINLY

A route parameter is a claim the visitor makes, not a fact the server established. Anyone can type any address. So a channel keyed on a route parameter alone can be named by any visitor who can reach the page, and no cleverer function signature fixes that — the problem is in what the caller is allowed to assert, not in how it is asserted.

The practical consequence: the chat room in the starter project is writable by anyone. That is correct behaviour for a demonstration and it should never be described as secured.

CHAPTER 22

SANDGATE — running other people's code

A Tier B render runs npm packages inside the JavaScript engine on the server. Those packages are code nobody at Albedo has read. SANDGATE is the answer to what happens when one of them misbehaves.

A JavaScript engine has a *realm* — its global environment, its built-in objects, everything a running script can see and change. If a package modifies something in there, every later request on that engine inherits the modification. That is the whole attack surface, and it is also the whole accident surface.

The primitive is realm rebuilding: throw away a poisoned realm and construct a fresh one. Proving that works, and works without leaking memory, was the first gate.

But a rebuilt realm is empty, and the caller is left to remember everything that had been registered in it. So SANDGATE keeps a **ledger** — every registration the engine has performed, in order — and a confinement is a rebuild plus a replay of the ledger. The caller knows nothing.

The ledger stores the *compiled script* rather than the original source. The original has to go through parsing, type stripping and lowering before it can be evaluated, and that work is deterministic in the source — so replaying it per request would be pure waste. Keeping the exact string that was evaluated makes replay a pure engine operation and keeps the compiler off the request path entirely.

Bytecode, and the three properties that make it safe

Re-registering 55 Radix modules through the ordinary path was measured at **4.415 ms**. Through a precompiled bytecode path it was **0.650 ms** — 6.8 times faster, and 71% of a confined request's total cost.

Loading bytecode into QuickJS is an unsafe operation, because the engine will execute whatever byte string it is handed, and malformed bytecode is arbitrary behaviour inside the very realm SANDGATE exists to confine. Three properties make it affordable here, and all three are load-bearing:

1. **The cache never leaves the process.** An on-disk cache would be a new integrity boundary — a tampered file would be code execution. Keeping the bytes in memory, produced microseconds earlier, removes that boundary rather than defending it.
2. **It is keyed by the script's content hash,** so a development reload that changes a module simply misses and recompiles.
3. **Version lock is automatic.** Bytecode is tied to the exact engine build, and because nothing is persisted, a rebuilt binary can never meet a stale artifact.

There is a further subtlety. Bytecode compilation forces strict mode, and a script that runs today in the older permissive mode may behave differently under it — an assignment to an undeclared name throws, top-level `this` becomes undefined. So the split is deliberate: **first registration always takes the ordinary path**, so the semantics an author sees are untouched whatever they wrote. Only *replay* uses bytecode. A script that refuses to compile is remembered and replayed as source forever after — a slower engine, never a broken one. There is an environment switch to disable the fast path wholesale.

The gate that asked the wrong question

One of the five SANDGATE gates set out to prove that a confined realm could not forge a capability — could not construct a reference to something it was not given. The investigation found that every path by which a forging attempt could travel went through *serialisation* first. The property that actually holds is about integrity of what crosses the boundary, not about capability inside it. The gate was answered, but it was not the question that had been asked, and the write-up says so.

CHAPTER 23

SHUTTER — rate limiting that knows what it is protecting

Rate limiting is the mechanism that stops one caller hammering an endpoint. Before SHUTTER, it did not exist anywhere in the tree, and an unauthenticated login endpoint is where that stops being theoretical.

The core algorithm is a standard one, GCRA, and there is a good crate available that implements it. Three layers above it are the reason SHUTTER exists anyway, and none of them are expressible without owning the compiler and the router.

- **The weight is derived, not typed.** A path-and-address limiter cannot know that this request is a cached render and that one is a write fanning out to five hundred channels, so every number it enforces is a guess. Albedo's weight is a by-product of facts the build already established.

- **The subject is the principal, not the address.** An IP address is a proxy for identity that carrier-grade network address translation and cloud egress made a poor one. With authentication in place, the limiter can ration the actor.
- **The table degrades rather than exhausting.**

The attack that kills a naive limiter, and why the fix is free

The thing that takes down a keyed rate limiter is not volume. It is **cardinality**: spray requests from a million distinct addresses and the table that was protecting you becomes the thing that runs the machine out of memory. A limiter whose memory an attacker controls is a liability wearing a mitigation's clothes.

Two properties make the fix cost nothing.

First, **a replenished cell is indistinguishable from a cell that does not exist**. The algorithm's state saturates at "no debt", so evicting an entry whose allowance has fully recovered loses exactly nothing — recreating it yields identical decisions forever after. Eviction is therefore keyed on replenishment, never on age or recency, because dropping an *indebted* entry would forgive debt, which is the one direction that favours the attacker.

Second, **overflow makes the limiter stricter rather than weaker**. When the exact table is full, keys fall back to a fixed-size array of shared cells addressed by hash. Under a cardinality attack the attacker's own keys collide with each other and throttle each other harder, while memory stays constant. The degraded mode is more aggressive than the exact one, which is the correct direction for a mechanism to fail in.

The cost is that a legitimate caller can share an overflow cell with an attacker during an attack. That is a real cost, stated rather than hidden. The alternative on offer is the process dying.

There is no Redis and no separate limiter service. The limiter is in-process and lock-free: no network hop on the request path, no shared store to be a single point of failure, and no question about what to do when the limiter is down, because there is nothing separate to be down. The trade is that limits are per-process, so a deployment of N instances admits up to N times the configured rate. For a login endpoint, a bounded multiple of a tight limit is worth more than an exact limit that can fail open when a cache is unreachable.

CHAPTER 24

The three refusals

Albedo refuses to finish a build in three separate ways, and each one exists because of a specific failure that used to be silent.

Preflight

`src/preflight.rs` is a small module with a large history. Every check in it closes a failure where an application built clean, booted clean, and served a perfectly healthy response with something missing. Each was found by taking the starter project the tool generates, breaking it in one specific way, and looking at what the tool said.

It said nothing. Five times.

Check	What used to be silent
channel names	a mistyped channel name rendered an empty list, forever
route main exports	a page with no main export served only the surrounding layout
form actions	a form naming an action that did not exist was a dead end at click time
npm imports	an uninstalled package dropped the page's content
deferred module loads	a deferred package load shipped verbatim to the browser and threw there

The shape repeats across all five: a fact the compiler had already computed, with no consumer.

These checks were originally written at boot, because boot is where the compiled project, the database schema and the served route set first meet. That left `albedo build` — the thing continuous integration runs — passing an application that `albedo serve` would refuse. A broken build shipped and failed at container start instead of in the pipeline that produced it. Writing them in two places would mean writing them twice, and two spellings of a rule disagree on exactly the inputs that matter. So they live in one module, take the facts as arguments, and are called by both.

The tier budget

A file called `tier-budget.toml` sets ceilings: how many components a route may fan out to, how many kilobytes a Tier B route may weigh, how large a single island's wrapper may be. `albedo build` fails when a ceiling is exceeded, and names the top contributing components so the fix is obvious from the pipeline output alone.

The reasoning is that tier classification is the load-bearing contract, and without a ceiling a careless import quietly promotes a component up the ladder and the cost only shows up in production. The budget makes the promotion visible at commit time.

`albedo doctor`

The third is a report rather than a gate: what the build already knows about itself. Which routes exist, which are gated, which read what data, and how each read is reachable.

One organising rule governs it: **every check is a derivation, never a document**. Doctor is allowed to report things the compiler established for its own reasons, and is forbidden from carrying a list somebody maintains by hand — because a hand-maintained list drifts from the system silently, which is the exact failure mode the tool exists to remove. Shipping one inside the tool would be the joke telling itself.

What that buys is that doctor cannot be *wrong* about a built project. Only incomplete.

CHAPTER 25

npm — how three million packages get in

This was, in the words of the research file, the single largest unknown in the business. If Albedo cannot run the packages real applications depend on, none of the rest matters.

So on 3 August 2026 somebody measured it. Take five real projects' declared dependencies, install each into its own folder, and run every single one through Albedo's loader.

92 of 108 packages loaded. 85.2%. And every one of the 16 failures fell into four causes, of which twelve were a single cause: there is no substitute yet for Node.js's own built-in modules. `axios` wants `util`. `stripe` and `bcryptjs` want `crypto`. `handlebars` and `qrcode` want `fs`.

Two of the remaining four are correct refusals — packages with no root entry point, which Node.js also refuses. One is a package that publishes a broken build. One needs a code-generation step that had not been run.

So the real figure is **92 of 104 genuine candidates, 88.5%**, and closing one bounded, well-understood piece of work takes this sample to 100%.

WHY THIS MEASUREMENT MATTERED MORE THAN ITS NUMBER

The feared answer was that the ecosystem is a long tail of a thousand distinct incompatibilities, each requiring its own investigation, with no end.

It is one gap with a known shape — a table mapping built-in module names to host functions, exactly as every bundler since browserify has done. And the failures cluster in *server-side* packages, which is precisely the surface the external-services and authentication work already sits on.

Tree shaking, and where the bytes actually were

A package usually contains far more than any one application uses. *Tree shaking* is the process of following what is actually referenced and discarding the rest.

Shaking `lucide-react` — an icon library — down to a single icon took it from **848,231 bytes** to **156,991 bytes**. A good result, and then somebody looked at what was left.

Of that 156,991 bytes, 3,507 was lucide's own code. Two point two percent.

The other 97.8% was React itself — dragged in twice, in both a development and a production build — plus `prop-types` at 34 kB, `react-is` at 13 kB, and `object-assign`. All of it pulled in by the *package's* own import of React, which, unlike an application's, was bound to nothing.

The fix binds those to the client runtime Albedo already ships, and collapses the development-versus-production branch so only one arm of each remains reachable. The test that pins this asserts that React never appears as a bundled chunk, that a development build never reaches a browser, and that the client-shaped bundle is under a quarter of the server-shaped one — a step change rather than a trim.

One chunk per package, unioned across the build

Each npm artifact is assigned to the chunk of the package that *owns* it, read off its own record key, rather than the package that happened to demand it. So `prop-types`, pulled in by two different libraries, is emitted once for the whole build.

The trade is stated in the source: demand is unioned across every island, so a page needing one icon downloads whatever icons the rest of the site needs too. That is one URL per package, content-hashed, cached once for the whole site and across deployments that do not change it — against per-route chunks that would re-transfer the overlap on every navigation.

The residual, which you should know about

There is an open gap. A package that loads a module *dynamically*, at the moment it needs it rather than at the top of the file, currently ships that instruction verbatim to the browser, where it throws. That is a runtime failure in a visitor's browser rather than a build error, which is the wrong side of the line, and it is ranked as the highest-priority correctness item outstanding.

CHAPTER 26

The commands

The entire surface a developer touches is one binary and ten verbs.

Command	What it does
<code>albedo init</code>	create a new application — the starter project, ready to run
<code>albedo dev</code>	the development server, with live reload and an incremental build
<code>albedo build</code>	compile for production, run every refusal, print the tier report
<code>albedo serve</code>	build and run the production server
<code>albedo ship</code>	build and configure a deployment target — Vercel, Docker, Fly, or a static export
<code>albedo files</code>	serve a folder of static files
<code>albedo budget</code>	check the tier budget on its own, in readable or machine form
<code>albedo doctor</code>	report what the build knows about itself
<code>albedo demo</code>	draw the instrument panel with synthetic signal
<code>albedo completions</code>	print shell completions

THE ONE COMMAND WHOSE NUMBERS ARE FABRICATED

`albedo demo` draws a beautiful dashboard, and **every number on it is invented**. It exists to show the shape of the instrument panel.

The rule the codebase enforces is that `demo` and `dev` are different verbs precisely so that this can never be ambiguous: *if a number appears under ``dev``, something measured it*. Never cite a figure from `demo`, in a deck, a demo, or a conversation.

What `albedo build` prints is the tier report — how many components landed in each tier, the measured byte count of every island that compiled, the npm chunks and their sizes, and the shared runtime. Every field on it is measured from the build's own output.

There is no Node.js to install, no bundler to configure, no package manager step for the framework itself, and no daemon. One binary, built from 59 shared declared Rust dependencies, containing a compiler, a JavaScript engine, an HTTP server, a database driver and a rate limiter.

Speed

The longest part, because speed is the thing you will be asked about and the thing most easily faked. Every number here has a date, a machine, and a file it came out of.

CHAPTER 27

Four clocks, and knowing which one is being talked about

Almost every argument about web performance is two people measuring different things. Albedo has four clocks and they behave differently, so before any number is quoted it is worth naming which one it belongs to.

Clock	What it measures	Who pays
Build	compiling the whole application	the developer, on every save under <code>albedo dev</code>
Boot	starting the server and priming it	the deployment, once
Request	one visitor asking for one page	every visitor, every time
Update	one change reaching every open browser	everyone connected, on every write

The optimisation ledger in the repository ranks work by that last column, and the rule is written down: **cost per viewer beats cost per write beats cost per boot**. A change that saves half a millisecond on every request outranks one that saves a hundred milliseconds at startup, because startup happens once and requests happen forever.

There is a second rule beside it, and it is the more useful of the two: **measure the shape, at two sizes, and compare the ratio**. A single number tells you how long something took. Two numbers at two sizes tell you what happens when the application grows, which is the only thing that matters. Both of the optimisations that landed in the 1 September 2026 sweep were invisible at one size, and one of them had been sitting unexamined for a month for exactly that reason.

CHAPTER 28

The language, and what it bought

Albedo is written in **Rust**. That choice pays in four places, and it is worth being able to say what they are rather than gesturing at “Rust is fast”.

No garbage collector on the request path. Languages like JavaScript, Java and Go periodically pause to reclaim memory. Those pauses are usually short and are occasionally not, and they land unpredictably — which is why tail latency in those systems is worse than average latency by more than people expect. Rust reclaims memory at compile-determined points, so there is no pause to schedule.

One binary. The compiler, the JavaScript engine, the HTTP server, the database driver, the rate limiter, the browser-side runtime and the starter project all ship inside a single executable file. The starter project and the client runtime are literally embedded into the binary at compile time. There is no dependencies folder for the framework, no runtime to install on the server, and no version skew between the tool that built the application and the tool that serves it.

Real parallelism. Node.js runs one thread. The Albedo compiler analyses components across cores, the server renders across a pool, and the runtime routes work across four lanes. Whether to go parallel is itself a decision — `src/analysis/adaptive.rs` holds a function that answers exactly that, because coordinating four threads to do two milliseconds of work is slower than doing it on one.

A compiler that refuses. The project bans four categories of mistake outright, at compile time: unchecked unwrapping, deliberate panics, unchecked indexing, and silent numeric conversion. Every one of them is a way a running server crashes, and none of them can get past the build. This is not a speed property directly, but it is why the server can be a single long-lived process holding live connections rather than something that needs a supervisor to restart it.

CHAPTER 29

One request, itemised

The most demanding request the system serves is a page whose components come from npm — the case where a full JavaScript engine has to render, and where the realm has to be confined and rebuilt because third-party code ran in it.

On the development machine — a Ryzen 9 9900X with 32 GB — in release mode, against a real Radix UI project with 55 npm artifacts, that request costs **1.379 milliseconds**.

And it is not held as one number. It is decomposed, so that the next optimisation can be aimed rather than guessed at.

Term	ms	What it is
npm registration	~0.44	8.0 μ s $\times$ 55 artifacts, re-registered into the fresh realm
eager re-linking	0.389	the packages' own factory chains, re-run
prelude rebuild	0.325	reconstructing the realm's globals, from bytecode
render	0.076	the actual page

Measured by the SANDGATE cost suite in `tests/`, release, 30 August 2026 and after the 1 September sweep. Recorded in the optimisation ledger under `development-plan/`.

Rendering the page is five and a half percent of the request. Everything else is safety.

That is the honest shape of it, and it is a good thing to be able to say out loud. Confinement is not free. It costs about 1.3 milliseconds to guarantee that one visitor's request cannot leave anything behind in the engine for the next one, and the decision to pay it was made deliberately, with the number in hand.

A request with no npm involved costs **0.368 ms** on the same machine, because there are no artifacts to re-register and no package factories to re-run.

How it got there

Two changes, measured separately.

The first put the realm's prelude — the globals every render needs — onto precompiled bytecode. On 30 August the prelude rebuild fell from **1.631 ms to 0.373 ms**, a confined request with no npm from **1.673 to 0.415 ms**, and a confined Radix request from **2.320 to 1.110 ms**.

The attribution inverted on the way: the prelude had been 72% of a confinement and became 35% of a much smaller one. Which is exactly why the test's assertion is written against the *total* rather than against the ratio — an assertion on a ratio would have started failing the moment the optimisation worked.

The second, on 1 September, suspends the garbage collector for the duration of the realm rebuild and restores it afterwards. Confined Radix went **1.562 $\rightarrow$ 1.379 ms**, a 11.7% improvement; no-npm went 0.409 $\rightarrow$ 0.368. Render time unchanged.

WHY TWO BASELINES APPEAR FOR THE SAME THING

The bytecode measurement records a Radix request at 1.110 ms and the garbage-collector measurement starts from 1.562 ms. Both are real; other work landed between the two runs.

This is stated rather than smoothed over, because a document that quietly picks the flattering number from each run is how performance claims become fiction. The current figure is **1.379 ms**, and it is the one to quote.

CHAPTER 30

The real source of the speed: work that is never done

Everything in the previous chapter is engineering at the level of microseconds, and it matters. But it is not where the difference against a conventional framework comes from.

The difference comes from the work that is never scheduled at all.

Consider a page with a heading, a paragraph, a navigation bar, a live guestbook, and one counter button. In a conventional setup, all five of those components are compiled into the bundle, downloaded, parsed, executed and hydrated. The visitor's phone reconstructs all five in memory to make one button clickable.

In Albedo, the heading, the paragraph and the navigation bar are Tier A — they exist as text in the response and nothing else. The guestbook is Tier B — its markup is text too, and when somebody signs it, the change arrives as a hundred and seventy-nine bytes of instruction. The counter is Tier C, and its code is compiled, alone, into an island.

One component's worth of JavaScript for a five-component page.

Measured on real code that was written by strangers, 61.2% of components take that route. There is no configuration flag that makes a conventional framework do this, because the conventional framework does not know which components are which — it never read the whole application, so it cannot.

What is actually shipped, in bytes

There is a fixed cost. Every Albedo page loads a shared runtime — the thing that applies instructions to the page, intercepts links and forms, and manages the live connection.

Shipped on	Raw	Gzipped
every page (runtime + link/form handling)	83,351 B	25,662 B
a page with live data (+ the trunk bootstrap)	102,289 B	31,159 B

Measured 2 September 2026 against the files in `assets/`, which are unminified source. This is a per-application cost, paid once and cached, and it is separate from component code.

That fixed cost should be stated whenever the 61.2% figure is. A page with zero component JavaScript still loads a runtime, and honesty about that is cheap.

Against it, the variable cost. A page in a conventional framework carries every component it contains. A page in Albedo carries the islands and nothing else — and the build prints the measured byte count of each island, so the number is never an estimate.

CHAPTER 31

Measure the shape: a story about 427 milliseconds

This is the best optimisation story in the repository and it is worth telling properly, because it demonstrates the method rather than just the result.

The compiler needs to find the critical path — the longest chain of dependent work through the component graph. The original implementation walked the graph and tried every route, marking nodes as it went and unmarking them on the way back out.

That approach remembers nothing. It re-derives the same sub-answers over and over. And component graphs are full of *diamonds*: every place two components share a `Button`, a layout, or an icon is a point where two paths split and rejoin. With k diamonds there are two-to-the-power-of- k distinct routes through the graph.

Nobody noticed, because the starter project has about a dozen components. So somebody measured it at several sizes.

Nodes	Time
25	0.14 ms
37	1.64 ms
49	26.99 ms
55	106.94 ms

Nodes	Time
61	427.10 ms and it could not have reached 1,801 nodes at all

Measured in release mode by the critical-path shape benchmark. Four times slower for every six nodes added — the signature of enumeration.

Four hundred and twenty-seven milliseconds, on the build path, which `albedo dev` runs on every single save. A sixty-one component application is small. A real one is several hundred, and the growth is exponential.

It had been in the codebase for a month, and nothing was wrong with it at the only size anyone had run it at.

The fix stores, for each node, its best *successor* rather than its best path. A cached answer is then a single value rather than a copied list, and the path is walked out once at the end. The result is linear in the graph.

Sixty-one nodes: 427 ms → 0.02 ms. One thousand eight hundred and one nodes, which the old version could never have finished, complete in **0.56 ms**. The growth is now three times the time for three times the nodes, which is what linear looks like from outside.

THE RULE THIS PRODUCED

No intuition about node counts would have found this. The numbers look fine right up until they do not, and the transition happens between two sizes nobody thought to test.

Measure the shape at two sizes and compare the ratio. It is now the first rule in the optimisation ledger, and it is the reason the same sweep also found the garbage-collector win.

CHAPTER 32

The refutation we published

The garbage-collector change had a first version that seemed obviously better and was wrong, and the record of it is kept in the repository beside the version that shipped.

The reasoning was clean: the JavaScript context is discarded at the end of the request anyway, so any memory collection happening inside it is work with no consumer. Relax the collector for the whole engine and stop paying for it.

Measured on the realm rebuild alone, it was a 14.7% improvement. Then somebody measured the whole request.

	confine	render	request
shipped — suspended for the rebuild only	1.303 ms	0.076 ms	1.379 ms
relaxed engine-wide — refuted	1.267 ms	0.673 ms	1.941 ms

The rebuild got faster by 36 microseconds. The render got **777% slower**, and the request went from 1.379 to 1.941 milliseconds. Net worse, by a wide margin.

The premise was false, and it was falsely reassuring in a specific way: the heap does not belong to the request’s context, it belongs to the engine’s runtime, which outlives the request. Deferred collection during a rebuild is genuinely free because the rebuild allocates and discards in one burst. Deferred collection during a *render* means the render allocates into a heap that keeps growing and pays for it in cache behaviour.

Two other tempting optimisations are recorded as advised against, with the measurement that says so. Caching the eager re-linking step, which costs 0.389 ms, would cache exactly the thing confinement exists to discard — it caches the poison. And batching the journal writes only pays when a single pass stages several outbound calls at once, which nothing does yet, so it is recorded as deliberately premature.

The refutation is the more useful half.

OPTIMIZATIONS2.MD, ON THE GARBAGE-COLLECTOR
CHANGE

CHAPTER 33

The data structures, and one that was taken back out

Below the architecture there is a layer of ordinary, careful, low-level engineering. You do not need to be able to explain these, but you should know they are there and that each was a deliberate choice.

- **Columnar layout.** The compiler’s intermediate representation keeps one array per field rather than a list of records. Processors read memory in contiguous blocks, so scanning one field across ten thousand components touches ten thousand adjacent values rather than skipping through ten thousand scattered records.
- **String interning.** Tags, attribute names and text values are stored once and referred to by number. The wire format inherits this, so a page mentioning `div two` thousand times sends the word once.

- **A dirty bitmap.** One bit per component saying whether it changed since the last frame, updated in bulk with vector instructions, so the reconciler can skip whole regions without looking at them.
- **Four lanes.** Rendering and update routing are partitioned four ways, and the data layout is physically aligned to that partition — with a compile-time assertion that fails the build if the two constants ever disagree.
- **Binary search instead of a hash table.** The component-to-lane lookup on the hot path is a sorted array probed linearly rather than a hash map, because at that size a cache-friendly scan beats a hash computation.
- **Engines pinned to threads.** The JavaScript engine cannot be moved between threads safely. Rather than working around that, each engine owns a dedicated operating-system thread for its whole life, and a “checkout” ships a closure to that thread and waits for the result. Nothing crosses a thread boundary except the closure and its answer.
- **Warm on construction.** Each engine in the pool runs eight warm-up renders before announcing itself available, so no request is ever served by a cold engine.

The optimisation that was removed

There used to be a faster one, and it is instructive that it is gone.

Memory for a request was allocated from a bump region and reclaimed at the end by resetting a single pointer — constant-time, regardless of how much had been allocated. It is about as fast as reclamation gets.

It was only sound if nothing the engine still referenced lived in that region at the end of the request. QuickJS interns long-lived internal structures — object shapes and property-name strings — through the same allocation path, *during* the request, keyed off the shapes and names it happens to see. Those stay reachable from engine-global tables across requests. A wholesale reset freed memory that was still live.

It surfaced the moment a route received per-request values whose *shape* the warm-up had not already created — a page with a parameter in its address, whose values object has a shape the empty warm-up never produced. The next request that reused the dangling structure crashed.

There is no general way to pre-create every shape an arbitrary application will produce, so the fast version could only ever have been patchwork. The persistent region survives for one-time start-up state that genuinely never frees; request memory is handed back to the engine, which tracks each block’s true lifetime. The comment in `src/runtime/arena.rs` recording this is one of the longer ones in the codebase, and it exists so that nobody reintroduces the fast version in two years’ time.

CHAPTER 34

The size of the change, not the size of the thing

If there is one idea that unifies the performance work, it is this: the cost of an update should be proportional to how much changed, and never to how much there is.

It shows up in four places, and each was a separate piece of work.

- 1. The wire.** Appending one row to a 2,002-row guestbook sends 179 bytes rather than 126,023.
- 2. The DOM.** A keyed delta names the row that changed, so the rows that did not change keep their actual page elements — and therefore keep focus, text selection, scroll position, and any animation in flight. A full-list repaint loses all four.
- 3. The database read.** A partitioned collection emits an index on the partition key, so reading one chat room costs what the room costs rather than what the whole table costs.
- 4. The build.** The incremental cache re-analyses only the file that changed and what depends on it.

The partition work has a measured justification too: sending a whole partitioned collection through the engine rather than the affected partition alone was measured at **4.9 times** a small room's write cost, before it was fixed.

And there is an honest limit. The compute side of the delta pipeline — recomputing what a view should now contain — is still proportional to the size of the collection on every write. The wire and the page reconciliation are proportional to the change; the recomputation is not. The plumbing for a true incremental engine is built, the algebra that would make it incremental is not, and the repository says so in plain words rather than implying otherwise.

CHAPTER 35

What is still slow

Four things, ranked, with the measurement that ranks them.

1. Registration cost grows with the whole application, not the page

This is the largest remaining item, and it is first because it is the only cost on the list that *grows with how successful the application is*. Everything else is a constant.

Rebuilding a realm re-registers every npm artifact in the application — not the ones this page uses. The slope was measured at **8.0 microseconds per artifact**.

npm artifacts in the app	Registration cost	Status
55 (measured)	≈0.44 ms	the shipped case
500 (projected)	≈4 ms	projection from the measured slope
2,000 (projected)	≈16 ms	projection

The slope is measured at 55; the protocol for the fix requires confirming it is still linear at 500 before designing on it.

The fix is to load an artifact only when it is first needed. The obstacle is not the loading — it is that each artifact registers both a factory and a name alias, and the alias lives inside the very script text a lazy load would skip, so imports would stop resolving. The shape of the answer is known: register the small flat alias table eagerly, load the factory on first use.

2. The prelude, at 0.325 ms per request

What remains after bytecode is *executing* rather than parsing, so the only way to remove it is to snapshot a fully built realm and restore it. Native QuickJS has no such interface, which means running the engine compiled to WebAssembly and paying that interpreter's overhead on every render forever. The protocol written down for this is to measure a WebAssembly render against a native one **first** — if the interpreter tax exceeds 0.325 ms, the idea is dead and the experiment cost an afternoon.

3. External calls hold a worker

An outbound HTTP call inside a handler was measured at **403.9 ms** of round trip in one gate. The engine is released across that boundary rather than held, which is the property that keeps a slow upstream from consuming the server — but three independent calls in one handler still cost four passes and three sequential round trips, because there is no batching. That work is designed and not built.

4. Continuous integration is not the gate people assume it is

This one is not a performance problem but it belongs on a list of honest weaknesses. The pipeline shows green, and its runs complete in thirty-five to ninety-six seconds — which is not long enough to be running the work the badge implies. The real gate is running the full test suite locally, and that is what the project treats as authoritative.

There is a related hazard worth knowing about, because it is the kind of thing that produces a false confident statement. A crash in the test suite truncates the run, and a truncated run reads as a pass. The project's workaround is to run without early exit and check the process's exit status explicitly.

The underlying crash is an intermittent one in the database driver's teardown when two database instances are being torn down at once. It is rare, it is understood, it has three known sites, and the operating rule is one teardown in flight per process.

A projection is not a measurement.

THE COMMENT AT THE TOP OF THE SANDGATE COST
SUITE

That sentence sits above the test that replaced an earlier estimate of 2.68 ms with a measured 1.11 ms. The earlier figure had been arrived at by applying a ratio measured on one part of the work to a total that also contained parts the ratio never covered — and a real product decision had already been taken on it. The decision survived, with room. It is kept as a caution rather than as a victory.

What It Means

The honest ledger, the competitive picture, and everything you need to hold a conversation about this without a laptop open.

CHAPTER 36

The developer experience, briefly

Part Three was about the machine. Two paragraphs on what it is like to use it, because you will be asked.

`albedo init my-app` writes a working application: a landing page, a live guestbook backed by a real database, a chat room with per-room channels, a sign-in page with real password authentication, and a counter that becomes an island. Seven components and routes, one stylesheet, one config file. No install step for the framework.

`albedo dev` starts a development server with hot reloading — a save re-analyses only what changed and pushes the difference to the open browser — plus an error overlay that appears in the page when something fails, and a terminal dashboard showing tier counts and live request timings. There is a browser-based inspector too, which draws the component graph and a per-lane utilisation heatmap, and which warns the operator when the measured render-cascade ratio drifts from what the compiler assumed.

A rule was learned there and is worth stating: **if a number appears under ``dev``, something measured it**. The synthetic dashboard lives behind a different verb for that reason.

CHAPTER 37

The honest ledger

Everything below is true, is written down inside the repository, and should be said before somebody discovers it.

Built and proven

- The three-tier classifier, measured against 1,398 real components from five projects that had never heard of us.

- The compiler pipeline end to end — scan, parse, graph, analyse, decide, schedule, emit, bundle, refuse.
- Tier B server-owned live state, with keyed deltas on the wire.
- FORGE: declared collections, emitted schema, seeds, drift refusal, atomic reservations, post-commit fan-out.
- PHOSPHOR: one connection per browser profile, server-authorised subscriptions.
- PRISM: minted channel identities, partitions, and the resolution rule in one module.
- APERTURE: outbound calls, response cache, coalescing, egress policy enforced at resolution, and a durable journal that brackets each call.
- AUTH: password sign-in, sessions, Argon2id, four emitted tables, identity as a channel key.
- SANDGATE: realm confinement, the registration ledger, in-process bytecode replay.
- SHUTTER: derived-weight rate limiting with bounded memory that degrades stricter.
- Preflight, tier budget, and doctor — three ways to fail before a visitor does.
- npm loading at 88.5% of real candidates, tree shaking, per-package chunks, React externalised.

Designed and not built

- Passkeys, OAuth, and delegated identity. The password provider works; the rest is a declaration format and no implementation.
- Batching independent outbound calls. Three calls in one handler still cost three sequential round trips.
- Lazy npm artifact loading — the largest measured headroom on the request path.
- A shim table for Node.js built-in modules, which would take the measured npm sample from 88.5% to 100%.
- A true incremental compute engine. The wire is proportional to the change; the recomputation behind it is not.
- CITRINITAS, the self-tuning layer: telemetry and a report, with automatic application deliberately excluded.

Known weaknesses

- A package that loads a module dynamically ships that instruction to the browser, where it throws. A runtime failure in a visitor's browser rather than a build error — the wrong side of the line, and the top-ranked correctness item.
- A channel keyed on a route parameter alone is nameable by anyone who can visit the page. The starter project's chat room is writable by anyone and must never be described as secured.

- The continuous-integration badge is not doing the work its green implies. The real gate is the local suite.
- An intermittent crash in the database driver’s teardown, understood, rare, with an operating rule around it.
- Registration cost is linear in the whole application’s npm surface rather than the page’s.

Two claims that were tested and failed

A design document once claimed that a particular ordering property held on both the server and the browser, and cited a neighbouring mechanism as proof. Investigation found the property holds only because a specific transform maintains it. The lesson written down was: **distrust a claim proved by a neighbour.**

And the escape-hatch design — the story about what happens when an application needs to run arbitrary code — had two headline claims falsified by a probe written to test them. The realm turned out to be shared and unfrozen, and the effect stream carried no provenance. Part of that has since been closed by SANDGATE; the record of the falsification stayed.

CHAPTER 38

Where this competes

The instinctive comparison is Next.js, and it is the right comparison for the wedge — “your site ships almost no JavaScript, and you did not have to decide which parts” is a sentence a Next.js user understands immediately.

It is not, though, where the real competitive ground is. Next.js is a rendering framework. Albedo also emits the backend: the tables, the queries, the live channels, the sessions, the rate limiter, the outbound-call journal. The systems it genuinely overlaps with are the reactive backend platforms — Convex, Cloudflare’s stack — the ones whose pitch is that you declare your data and the platform handles the rest.

What Albedo has that they do not is that the *interface* is inside the same analysis. A conventional reactive backend knows your data model and knows nothing about your components. Albedo derives the tier of a component from whether its state reaches the database, which is a fact that only exists if one system can see both halves.

Two sentences, in order, are the positioning: **the wedge is “no backend”; the moat is “it tunes itself”**. The first gets attention. The second is what would be hard to copy, and it depends on the compiler owning the whole picture.

Glossary

Alphabetical. Every term you are likely to hear in a room where Albedo is being discussed.

Term	Meaning
Action	A function the author declares that runs on the server when a form is submitted or a handler fires.
APERTURE	The subsystem for talking to outside services. Reads are cached channels; writes are journalled with derived idempotency keys.
AST	Abstract syntax tree. The structured form of source code, like a sentence diagram, that a compiler reasons about.
AUTH	The subsystem for accounts. Four tables, emitted from a declaration; identity becomes a channel key.
Bundler	The tool that squashes many source files into few downloadable ones. Albedo has its own, in <code>src/bundler/</code> .
Collection	A declared table in FORGE. Declaring it emits the schema, the query, the seeds, and a live channel.
Component	One named, self-contained piece of an interface — a header, a button, a list.
Confinement	Throwing away a JavaScript realm after untrusted code has run in it, and rebuilding it from a ledger. See SANDGATE.
Critical path	The longest chain of work that must happen in order. It sets the floor on how fast a build can be.
Delta	A description of what changed rather than what now is. Albedo's deltas are signed — plus one for an insert, minus one for a retraction.
Effect profile	Four flags per component: does it hold memory, wait, touch the outside world, cause side effects. The input to tiering.
FORGE	The backend subsystem. Data, from a declaration.
GCRA	The rate-limiting algorithm SHUTTER uses. Its state saturates at “no debt”, which is what makes eviction free.
Hydration	The browser downloading a program and rebuilding a page it can already see, in order to make it interactive. The central cost Albedo attacks.
Intern table	A numbered dictionary of repeated strings, sent once and referred to by number thereafter.

Term	Meaning
Island	A single interactive component compiled on its own and surrounded by static markup. What a Tier C component becomes.
JSX	The notation React components are written in — markup embedded inside code.
libSQL	The storage engine underneath FORGE. A fork of SQLite.
Manifest	The single artifact recording every decision the build made. The server reads it and nothing else.
Node.js	A way to run JavaScript on a server. Albedo does not use it and does not require it to be installed.
npm	The shared warehouse of roughly three million JavaScript packages.
Opcode	One instruction on the wire — create this element, set this text, remove this row.
PHOSPHOR	The connection layer. One line per browser profile; tabs share it.
Preflight	The set of checks that refuse a build for failures that used to be silent.
Principal	The authenticated identity behind a request. A compiler-known value, which is what lets authorization be derived rather than written.
PRISM	The channel-identity subsystem. Partitions, minted names, and one resolution rule.
QuickJS	The small complete JavaScript engine embedded in the binary. Written by Fabrice Bellard.
Realm	A JavaScript engine's global environment. The thing confinement discards and rebuilds.
React	The dominant component library, made at Facebook.
Request / Response	The two halves of the conversation between a browser and a server.
Rust	The language Albedo is written in. No garbage collector, strong compile-time guarantees, real parallelism.
SANDGATE	The confinement subsystem — running other people's code without letting it persist.
SHUTTER	The rate limiter. Weight derived from what the endpoint does; memory bounded; degrades stricter.

Term	Meaning
SSR	Server-side rendering. Producing finished markup on the server rather than in the browser.
SWC	The Rust parser Albedo reads source with. The same one Next.js uses internally.
Tier A / B / C	The three destinations for a component: markup with no state, markup with server-owned state, or a client island.
Topic / channel	A named stream of live data. Its name is minted by the compiler and cannot be typed by an author.
Tree shaking	Discarding the parts of a package nothing actually references.
Z-set	A signed multiset. The mathematical shape of “what changed”, used for FORGE deltas.

CHAPTER 40

The map of the repository

If somebody opens the project in front of you, this is what they are looking at. 132,589 lines of Rust across 215 files, plus the browser runtime, the starter project, and the test suite.

The compiler crate — `src/`

Path	What lives there
<code>lib.rs</code>	The facade. <code>RenderCompiler</code> , and the tier table in the crate documentation.
<code>effects.rs</code>	The tier decision. The product, in one function.
<code>parser.rs</code>	SWC-powered ingestion. Also where the “reads the principal” and “imports npm” facts are read off the source.
<code>scanner.rs</code>	Walks the project directory and produces parsed components. Lenient or strict.
<code>graph.rs</code>	The component dependency graph and its validation.
<code>estimator.rs</code>	Weight estimation — how big is this component likely to be.
<code>incremental.rs</code>	The build cache. File hashes, cached analyses, invalidation reasons.
<code>preflight.rs</code>	The five checks that used to be silent. Called by both build and boot.

Path	What lives there
<code>types.rs</code>	Shared types and the error enum.
<code>analysis/</code>	Effect analysis, topological sorting, the critical path, and the adaptive decision about whether to parallelise at all.
<code>transforms/</code>	Eleven passes over the syntax tree — hooks, events, forms, actions, links, shared slots, CSS modules, stable ids, deferred children, workflow lowering.
<code>ir/</code>	The intermediate representation: columnar storage, opcodes, the wire encoding, action envelopes, and a locked conformance frame.
<code>manifest/</code>	Building, validating and serialising <code>RenderManifestV2</code> , including route authorization metadata.
<code>routing/</code>	File-based routing — the folder structure becomes the address structure.
<code>hydration/</code>	The island plan, the payload, and the script tags that carry them.
<code>bundler/</code>	Classify, plan, rewrite, emit. Plus npm resolution and shaking, client-side npm chunks, vendored and precompiled artifacts, and the static slice.
<code>runtime/</code>	The largest area. See the table below.
<code>forge/</code>	Substrate, libSQL, in-memory double, reservations, drift, delta, write path, type generation, bindings, skeleton.
<code>aperture/</code>	Egress, cache, client, transport, workflow driver, journal, durable ledger, reader, refresh loop, type generation.
<code>auth/</code>	Principal, declaration, schema, session, store, password hashing.
<code>shutter/</code>	Cost classification and the GCRA cells.
<code>budget/</code>	The tier budget: configuration, evaluation, bundle-byte report, formatting.
<code>doctor/</code>	The reach matrix — what the build knows about itself.
<code>conformance/</code>	Cross-renderer comparison and normalisation.
<code>dev/</code>	The development contract, the showcase, and four benchmark harnesses.
<code>bin/</code>	The <code>albedo</code> command-line tool, its terminal dashboard, and the benchmark binary.

Inside the runtime — `src/runtime/`

Area	What it does
The QuickJS engine	The embedded JavaScript engine. 6,633 lines — the largest file in the project.
The compiled project	The compiled project: components, handlers, slot allocation, action invocation.
The pure-Rust evaluator	The pure-Rust evaluator that bakes Tier A markup.
Route rendering	Route rendering, the module registry, and render timings.
React host modules	The host modules Albedo provides itself, so packages' own React imports bind to something.
The topic registry	The topic registry and fan-out. 2,598 lines, and the home of the snapshot rule.
The channel resolver	PRISM's resolver. One function, three callers.
The slot store	Per-session reactive state, keyed by session and slot.
Session identity	Session identity.
Confinement	SANDGATE — the registration ledger and the bytecode cache.
The allocator	The allocator. Persistent warm-up region plus engine-managed request memory.
The async bridge	The seam between handler invocation and the async world.
The four-lane pipeline	The four-lane runtime pipeline: reconcile, tick, emit.
Lane assignment	Lane assignment and cross-lane dependencies.
The scheduler	Frame scheduling.
The frame tick	One tick of the runtime, and its metrics.
The opcode emitter	Turning changes into opcode frames.
The dirty bitmap	One bit per component, bulk-updated.
The hot set	Render priority — Low, Normal, High, Critical — capped at 32 entries.
The static slice	Identifying the genuinely static portion of a component.
The transport	The multiplexed transport and its stream router.
Inter-lane messaging	Inter-lane messaging.

Area	What it does
Core affinity	Pinning the worker pool to cores.
The engine seam	The engine trait and the bootstrap payload.
Node built-ins	What exists of the Node.js built-in surface.
JSX attributes	Attribute naming rules, including the SVG cases.
Form results	The shape a form submission returns.
Render observers	Observation hooks — render cascades at compile time, lane utilisation at runtime.

Everything else

Path	What lives there
<code>crates/albedo-server/</code>	The HTTP server: routing, streaming handlers, action dispatch, Tier B rendering, the engine pool, the PHOSPHOR lane, CSRF, the inspector, request timing.
<code>crates/albedo-node/</code>	Bindings that expose the compiler to Node.js as a native addon, for anyone who wants to drive it from an existing toolchain.
<code>assets/</code>	The browser-side runtime — client, reactive bindings, link and form interception, the transport bootstrap, the PHOSPHOR client, the binary decoder, the error overlay, hot-reload application. Embedded into the binary at compile time.
<code>scaffold/</code>	The starter project <code>albedo init</code> writes. Also embedded into the binary.
<code>tests/</code>	64 integration test files — SANDGATE's five gates, APERTURE's gates, renderer conformance, npm coverage and shaking, the tiering corpus probe, adversarial input, a crash-kill harness for FORGE, and more.
<code>benches/</code>	Seven benchmark suites: frame ticks, opcode encoding, cold start, first-paint bytes, hydration bytes, action round trips, PRISM partitions.
<code>benchmarks/</code>	The gating envelope. A regression policy of 15%, and named competitors to measure against.
<code>fuzz/</code>	Randomised input testing.
<code>installer/</code>	Distribution.

Path	What lives there
<code>development-plan/</code>	Forty-seven design and measurement documents, plus the decks, the legal drafts and the launch material. Not in version control — one disk, no remote copy. This is the single largest operational risk in the repository and it is not a technical one.

THE ONE ACTION ITEM IN THIS CHAPTER

The `development-plan/` folder holds roughly 300 kB of design documents, measurement records, strategy and legal drafts, and it is excluded from version control entirely. It exists on one machine.

Everything in Part Four came out of those files. Losing them would not lose the code, but it would lose every reason the code is the way it is.

CHAPTER 41

Five questions, and what to say

"So what is it, in one sentence?"

A compiler that reads your whole application before it runs, decides for each piece of the interface whether it belongs on the server or in the browser, and emits exactly what that decision requires — including the database, the live updates, and the accounts.

If they want it shorter: *it works out which parts of your site need to be a program, and only sends those.*

"How is that different from Next.js?"

Two differences. Next.js makes the developer mark which components run in the browser, by hand, file by file. Albedo derives it, from a compiler that has read everything. And Next.js is a rendering framework — Albedo also emits the backend, so the database, the live channels, the sessions and the rate limiting come out of the same declaration.

The number: **61.2% of components in real projects ship no JavaScript at all.** Measured on 1,398 components from five open-source applications that had never heard of us. Volunteer the caveat — on our own sample code it is 55.8% Tier A rather than 30.7%, twenty-five points of flattery, and we refuse to quote that one.

"Is it fast?"

Ask which clock they mean, then answer it.

For a visitor: the fastest page is the one that ships no program, and most of a real page qualifies. For the server: the hardest request the system serves — a page built from npm components, with the JavaScript realm confined and rebuilt so nothing leaks between visitors — costs **1.379 milliseconds** on our development machine, and we can itemise it into four rows. Of that, rendering the actual page is 0.076 milliseconds. The rest is safety we chose to pay for with the number in hand.

If they push: the best single illustration is the critical-path walk that went from **427 milliseconds to 0.02 milliseconds** at sixty-one components — and had been sitting there for a month because the sample project is too small to show it.

"What does it not do yet?"

Answer this one quickly and specifically; hesitating is worse than the content. Passkeys and OAuth are designed and not built. Independent outbound calls are not batched. Node.js built-in modules have no substitute yet, which is the one gap between 88.5% and 100% npm coverage on our measured sample. A package that loads a module dynamically currently fails in the browser rather than at build time, which is the wrong side of the line and is the top-ranked fix.

And then say the thing that makes the list credible: all of that is written down inside the repository, in the same files as the successes, with the measurement that ranks it.

"Why should I believe the numbers?"

Because the ones that were wrong are still in the files, next to the ones that replaced them.

A garbage-collector optimisation that looked like a 14.7% win was measured across the whole request, turned out to make it 777% slower in one term and net worse overall, and the refutation is kept in the ledger. A projection of 2.68 milliseconds was replaced by a measurement of 1.11, and the note above the test explains how the projection went wrong. A tier-classifier signal that seemed like a lever was measured, found to be true 99.6% of the time, investigated, and the structural reason it could never have been a lever was written down.

A system that only publishes its wins has not measured anything. It has been advertising.

. . .

That is the whole thing. A compiler that reads everything, three tiers, a backend that comes out of a declaration, a wire that carries the change rather than the thing, and a set of subsystems each named after the one idea it holds up.

If you remember one sentence from these pages, make it the one from Chapter 6 — because every other chapter is machinery for keeping it true.

*Albedo reads your entire application before it ever runs,
decides for each piece of the page where that piece should live,
and then emits exactly what that decision requires — and
nothing else.*